Requirements and Open Decisions

AI-Based Application for Balancing U-Shaped Assembly Lines (ILE 401)

Draft v0.1 - 9 October 2026 - Software and AI tracks

1. Purpose of this document

This is the starting point for the project. It lists what the web application must do (functional requirements) and how well it must do it (non-functional requirements) for the backend, the frontend and the AI track. Items that still need a team decision are marked **(D#)** and are collected in Section 7 with a suggested answer and space to write the outcome. After the meeting this document is updated and becomes version 1.0.

Priority key: M = must (the project fails without it), **S** = should (important, cut only if time runs out), **C** = could (first to be cut).

2. Project summary and decisions already taken

A web application where a user enters the tasks of an assembly process (name, time in minutes, precedence), the working time per day and the daily production. The system computes the cycle time, runs a Genetic Algorithm (GA) to assign tasks to stations in a U-shaped line, and shows the result as a U diagram with metrics, charts and short recommendations. Results are stored in a database.

- Genetic Algorithm is the core of the project. No simulation, no real-time control, no reinforcement learning.
- A small machine-learning layer produces text recommendations only. Using a neural network to warm-start the GA is dropped (future work).
- Stack: React frontend, FastAPI backend, Python GA, Supabase database.
- U-line rule: a task can be assigned when all its predecessors or all its successors are already assigned.
- Cycle time = working time per day / production per day. Minimum stations = ceil(total work content / cycle time).
- Benchmark data (Scholl ALB instances) is used for building and testing. Real factory data is used for validation only.
- Target is desktop browsers (tablet usable). Phones are out of scope.
- Team: project lead (backend, GA, integration), frontend engineer, AI engineer (ML recommendations), industrial engineering students (data and validation).

3. Backend requirements (FastAPI + Supabase)

Functional

ID	Requirement	Priority
BE-F1	Accept a problem instance: tasks (name, time, predecessors), working time per day, production per day, optional worker count, optional GA settings and optional random seed.	**M**
BE-F2	Validate every input and return clear field-level errors for: duplicate task names, non-positive times, unknown predecessors, cycles in the precedence graph, a task longer than the cycle time, task count over the limit (D2), production of zero or less.	**M**
BE-F3	Compute cycle time, theoretical minimum number of stations and suggested number of workers (D3).	**M**
BE-F4	Run the GA and return: stations with ordered tasks, the U-side of each task, metrics (efficiency, idle time per station, bottleneck stations), convergence history and recommendations.	**M**
BE-F5	Save every run (input, settings, seed, result) to Supabase. Provide list, get-by-id and delete.	**M**
BE-F6	Run long jobs asynchronously: start a run, then poll its status (D5).	**S**
BE-F7	Save, list and load reusable task datasets. Import from CSV and JSON.	**S**
BE-F8	Health endpoint and one consistent error format for all endpoints.	**S**
BE-F9	Export a run as JSON or CSV. Import Scholl benchmark files (D17).	**C**

Non-functional

ID	Requirement	Priority
BE-N1	Performance: a 50-task run with default settings finishes in about 30 seconds or less. Hard server-side timeout and cap on generations.	**M**
BE-N2	Security: request size limits, rate limiting, CORS restricted to the frontend origin, secrets only in environment variables, Supabase service key never in the client, row-level security on tables, HTTPS in deployment.	**M**
BE-N3	Reliability: invalid input never crashes the server. The same input and the same seed give the same result.	**M**
BE-N4	Maintainability: typed Pydantic schemas, separated layers (API, service, GA engine, persistence), auto-generated OpenAPI documentation used as the contract with the frontend.	**M**
BE-N5	Testing: unit tests for validation and metric calculations, at least one end-to-end API test, all run automatically on every pull request.	**M**
BE-N6	Operability: structured logging, documented way to run locally, public deployment on a free tier (D12).	**S**

4. Frontend requirements (React)

Functional

ID	Requirement	Priority
FE-F1	Task input table: add, edit and remove tasks, choose predecessors from existing tasks. Fields for working time per day and production per day.	**M**
FE-F2	Client-side validation that mirrors the backend rules, with inline messages.	**M**
FE-F3	Show cycle time, minimum stations and suggested workers live as the user types, with a worker override (D3).	**M**
FE-F4	Run button with a loading state and readable error messages.	**M**
FE-F5	Results view: U-shaped diagram, metric cards and a station table. Bottleneck and high-idle stations are visually flagged.	**M**
FE-F6	Charts: workload vs cycle time, idle time per station, worker workload, GA convergence.	**M**
FE-F7	Recommendations panel showing the ML text output.	**M**
FE-F8	Quick demo path: from the home screen, "Load example" then "Run" produces a full result in two clicks.	**M**
FE-F9	Run history: list, reopen and delete saved runs (depends on D4).	**S**
FE-F10	Import tasks from CSV or JSON. Collapsible advanced settings (population size, generations, seed).	**S**
FE-F11	Compare two runs, export diagram as PNG or PDF, precedence graph preview (D16).	**C**

Non-functional

ID	Requirement	Priority
FE-N1	Usability: a first-time user completes a run without instructions. Error messages say what to fix.	**M**
FE-N2	Layout: desktop-first, usable on a tablet. Phones are out of scope.	**M**
FE-N3	Performance: the interface stays responsive with 50 tasks (table, diagram and charts render quickly).	**M**
FE-N4	Quality: component library for a consistent look, API client generated or typed from the OpenAPI spec so the contract cannot drift (D11).	**S**
FE-N5	Security: no secrets in the client. User-entered text such as task names is escaped when displayed.	**M**
FE-N6	Compatibility: latest Chrome and Edge. Firefox if time allows.	**S**
FE-N7	Independence: the frontend can run fully against mock data so it never waits for the backend.	**M**

5. AI track requirements

5.1 Genetic Algorithm engine

ID	Requirement	Priority
AI-F1	Solution encoding and decoder such that every decoded solution is feasible by construction: no precedence or U-rule violation and no station above the cycle time.	**M**
AI-F2	Fitness function: minimise the number of stations, with a tie-breaker for smoother workload (D7).	**M**
AI-F3	Selection, order-preserving crossover, mutation, elitism and stopping criteria (maximum generations and stagnation).	**M**
AI-F4	Configurable parameters with sensible defaults, plus a random seed.	**M**
AI-F5	Record best and average fitness per generation for the convergence chart.	**M**
AI-F6	Baseline heuristic (for example largest task first with the U-rule) to compare against the GA (D8).	**M**
AI-F7	Correct handling of edge cases: single task, no precedence, pure chain, a task equal to the cycle time.	**M**
AI-F8	Benchmark harness: run the engine on Scholl instances and report the gap to the known optimum (D17).	**S**
AI-N1	Correctness: 100 percent feasible outputs, verified by an independent checker function used in the tests.	**M**
AI-N2	Reproducibility: a fixed seed gives an identical result.	**M**
AI-N3	Quality target: defined after the baseline exists (D8). No number is promised before that.	**M**
AI-N4	Isolation: pure Python module with no web or database dependencies, so it can be tested and benchmarked alone.	**M**

5.2 ML recommendation module

ID	Requirement	Priority
ML-F1	Take instance features and GA result metrics, and return a short list of human-readable recommendations (D9).	**M**
ML-F2	A rule-based fallback behind the same interface, delivered first. Recommendations always work, even if the model does not.	**M**
ML-F3	Reproducible training script. Training data generated by running the GA on synthetic instances (D10).	**M**
ML-F4	Model version stored with each saved run.	**S**
ML-N1	Latency: inference under one second, model loaded once at startup.	**M**
ML-N2	Honest evaluation: results reported against the rule-based baseline, even if the gain is small.	**M**
ML-N3	Robustness: a model failure never blocks the main result.	**M**
ML-N4	Explainability: each recommendation states the metric that triggered it.	**M**

6. Cross-cutting requirements

ID	Requirement	Priority
X-1	Data contracts: versioned JSON schemas for problem input, solution output and recommendation input and output, frozen at the end of Sprint 0 (D15).	**M**
X-2	Quality gates: protected main branch, pull request review by the project lead, automatic lint and tests on every pull request, a shared Definition of Done on every task (D13).	**M**
X-3	Documentation: README that lets a new person run the project in about 10 minutes, API spec, GA design document, decision records.	**M**
X-4	Data handling: real factory data is anonymised and kept out of the repository. Benchmark datasets are credited to their source.	**M**
X-5	Working method: two-week sprints, each ending with an end-to-end demo, a cut list agreed in advance and a feature freeze before the final stretch (D14, D16).	**S**

7. Decisions to discuss in the meeting

Each row has a suggested answer to start the discussion. Write the outcome in the right-hand column. The requirements above are then updated to match.

ID	Question	Options and suggestion	Affects	Decision / notes
D1	Which problem type: Type 1 only, or also Type 2?	Type 1: minimise stations for a given cycle time. Type 2 (minimise cycle time for fixed stations) adds a second objective and more testing. **Suggest: Type 1 only.**	GA, API, UI	
D2	Maximum number of tasks per instance?	Cap at 50 tasks, treat larger as future work. A cap bounds GA runtime and simplifies the diagram. **Suggest: 50.**	BE-F2, BE-N1, FE-N3	
D3	How are workers handled?	Option A: workers are suggested and the user can override, used for reporting workload per worker only. Option B: worker count also changes the GA objective. **Suggest: A.**	BE-F3, FE-F3, GA	
D4	Are there user accounts?	No login: runs are tied to an anonymous browser session. Login: per-user history, but adds authentication and row-level security work. **Suggest: no login for the MVP.**	FE-F9, BE-N2	
D5	Are runs synchronous or asynchronous?	Synchronous is simpler. Asynchronous (start, then poll) avoids a frozen page on longer runs. Decide after measuring GA time on 50 tasks. **Suggest: build async if a default run takes more than about 5 seconds.**	BE-F6, FE-F4	
D6	Which special constraints are supported?	Examples: tasks fixed to a station, tasks that cannot share a station, parallel stations. **Suggest: none for the MVP, list as future work.**	GA, schemas	
D7	What exactly does the fitness function optimise?	Primary: number of stations. Tie-breaker options: variance of station times, or maximum idle time. The formula must be written down once and used everywhere. **Suggest: stations first, then smoothness (variance).**	AI-F2, metrics	
D8	Baseline heuristic and success target for the GA?	Pick one simple baseline, run it on the benchmark set, then set the target from the result (for example optimal or one station above on most instances up to 50 tasks). **Suggest: agree the method now, the number later.**	AI-F6, AI-N3	
D9	What does the ML module predict or recommend?	Options: classify the main problem of a solution (for example a bottleneck station, high idle, imbalance) and map it to a recommendation text; or predict expected improvement. The AI engineer needs a precise definition. **Suggest: classification of the main issue with templated text.**	ML-F1	
D10	Where does ML training data come from and how much?	Generate synthetic instances, run the GA, label results with rules. Needs the GA output schema fixed first. **Suggest: a few thousand synthetic instances.**	ML-F3, X-1	

ID	Question	Options and suggestion	Affects	Decision / notes
D11	Frontend tooling: TypeScript or JavaScript, component library, chart library?	TypeScript catches contract mistakes but is harder for a beginner relying on AI tools. **Suggest: decide with the frontend engineer. Pick one library per need and do not mix.**	FE-N4	
D12	Where is the app deployed and who keeps it running?	Public HTTPS link for frontend and backend on free tiers. Free backends may sleep and take about a minute to wake: add a waking-up message or a keep-alive ping before demos. **Suggest: deploy a first version in Sprint 0.**	BE-N6	
D13	Repository and workflow rules?	One monorepo, protected main, feature branches, pull requests reviewed by the project lead, automatic checks. A Definition of Done checklist on every task. **Suggest: adopt as written.**	X-2	
D14	Sprint structure and meeting rhythm?	Two-week sprints with planning, a short sync two to three times a week (or a written update), a demo and a short retrospective at the end. Sprint 0 is week 1. **Suggest: adopt.**	X-5	
D15	When are the data contracts frozen?	The input, output and recommendation schemas are the agreement between all tracks. **Suggest: freeze at the end of week 1 and change only through a pull request that all tracks review.**	X-1	
D16	What is the cut list and when is the feature freeze?	Agree now what gets dropped first if time runs short: run comparison, exports, precedence preview, then ML polish. The core (GA, API, diagram) is never cut. **Suggest: freeze new features at the start of week 7.**	FE-F11, X-5	
D17	Which benchmark instances, and are import and export in scope?	Choose a set of Scholl instances of varied size up to 50 tasks, with published optimal values. Importing the benchmark file format is optional. **Suggest: use a fixed set of about 10 to 15 instances.**	AI-F8, BE-F9	
D18	Who owns and reviews what?	Project lead: backend, GA, integration, reviews and merges. Frontend engineer: UI tickets. AI engineer: ML module and training script. Agree the review turnaround time (for example within one day). **Suggest: adopt.**	All	

8. Next steps after the meeting

- Update this document with the decisions and publish version 1.0.
- Write user stories with acceptance criteria for all Must items.
- Draw the architecture and write the API and data schemas (Sprint 0).
- Set up the repository, automatic checks, Supabase project and a first deployed walking skeleton.